Programming Assignment #3 Pizza Shop

You have finally done it!!! You have graduated and created our own software development company. Your company just landed a contract to develop a Pizza shop management app. The client will pay you a lot of money if you can complete the project with all his requirements.

- The pizza shop sells two types of pizzas
  - Original
  - Deep Pan
- The Pizza shop offers 5 different toppings for pizzas
  - Bacon
  - Pineapples
  - Pepperoni
  - Mushrooms
  - Extra Cheese
- The toppings add to the cost of the pizza.
- Pizza takes different times to cook. The more toppings, the longer it takes.
- The Pizza shop only has a single branch.
- Each pizza order should be alerted (**notified**) when the pizza is finished cooking.

In this assignment, you will demonstrate your knowledge of the topics covered in Object Oriented Programming by making use of **Interfaces**, **Classes**, **Abstract classes**, **ArrayLists** and **Design Patterns** and/or other topics that may be suitable to complete the assignment.

---

1. You begin by creating the class **PizzaOrTopping**. This class is an **abstract** class that represents any pizza or topping. The **PizzaOrTopping** class has three (3) instance variables **protected String description**, **private boolean isFinished**, **private int orderNum**.

- Create a constructor that accepts **orderNum** as a parameter. The constructor must also
  - Set the **isFinished** instance variable to *false*.
  - Set the **description** variable is set to *"Unknown Pizza".*
- Create the accessors **getDescription(), getOrderNum(), getIsFinished()**.
- Create a method **finish().** This method sets the value of **finished** to *true*.
- Create a **toString**() method that simply returns the description.
- Create a public **abstract** method **long getCookingTime()**;
- Create a public **abstract** method **double cost()**;

2. You must now create classes that will represent the type of pizzas that the store sells. These classes are **sub-classes** of **PizzaOrTopping**. Each pizza type has two instance variables: **double cost**, **long cookingTime.** Create the classes as follows.

- Create a class named **OriginalPizza** that **extends PizzaOrTopping**
  - Create the instance variables for the class. {these are mentioned above}
  - Create a **super constructor** that accepts orderNum as a parameter.
    - Set **description** to *"Original Pizza\n"*.
    - Set **cost** to *3.75*
    - Set **cookingTime** to *10000*

- Implement the **getCookingTime()** method**.** This simply returns the cookingTime.
- Implement the **cost()** method. This simply returns the cost variable.

- Create a class named **DeepPanPizza** that **extends PizzaOrTopping**
  - Create the instance variables for the class. {these are mentioned above}
  - Create a **super constructor** that accepts orderNum as a parameter.
    - Set **description** to *"Deep Pan\n"*.
    - Set **cost** to *4.50*
    - Set **cookingTime** to *15000*
  - Implement the **getCookingTime()** method. This simply returns the cooking time variable.
  - Implement the **cost()** method. This simply returns the cost variable.

3. You've decided that the toppings of the pizza will be implemented using the **Decorator design pattern**.

- Create an **abstract** class named **Topping.** The **Topping** class should be a **sub-class** the **PizzaOrTopping** class.
  - Create a **super constructor**.
  - Create an **abstract** method **String getDescription()**.

4. Now create the different toppings as classes. These classes are **sub classes** of **Topping**.

- Create the topping class named **Bacon.** The class should have three (3) instance variables: **private PizzaOrTopping pizzaOrTopping**, **double cost**, **int cookingTime**.
  - Create a constructor that accepts a **PizzaOrTopping** as a parameter.
  - The constructor should:
    - Call **super** and pass the orderNum of the parameter {put hint here}
    - Set the instance variable **pizzaOrTopping**.
    - Set the **cost** to *0.75*
    - Set the **cookingTime** to *2000*
  - Create a **toString()** method that returns *"\tBacon\n"* {this is the name of the topping}
  - Create an accessor **getDescription()**. This should return the concatenation of the **pizzaOrTopping.getDescription()** and **toString()**.
  - Create an accessor **getCookingTime()**. This should return the concatenation of the **pizzaOrTopping.getCookingTime()** and **cookingTime.**
  - Create a method **cost()**. This should return the concatenation of the **pizzaOrTopping.cost()** and **cost.**
- Complete the other toppings classes by using the following table. These toppings classes should be identical to the Bacon class. {see table below}

ClassName	cost	cookingTime	toString
Pineapple	1.00	3500	\tPineapples\n
Pepperoni	2.00	2500	\tPepperoni\n
Mushroom	1.25	3000	\tMushrooms\n
Cheese	1.00	3000	\tExtra Cheese\n

5. You have decided that the Order and Oven classes will be implemented using an **Observer Design pattern**. The Order will observe the Oven and will be updated whenever a pizza is ready.

- Create the **Subject** interface and the **Observer** interface {review your notes}.
    - The Observer interface should have one method, **update(PizzaOrTopping pizza)**
    - The Subject interface should have the methods **registerObserver(Observer o), removeObserver(Observer o), notifyObservers()**

- Create the class **Oven**. The **Oven** class should **implement** the **Subject** interface. The **Oven** class has three (3) private instance variables **PizzaOrTopping finishedPizza, ArrayList<PizzaOrTopping> pizzas, ArrayList<Observer> observers.**
    - **finishedPizza** is used to hold the pizza object that is finished baking. Notified observers will access this.
    - Create a no-argument constructor. Within the constructor:
        - Create the **pizzas** ArrayList
        - Create the **observers** ArrayList.
    - Implement the method **registerObserver(…)** : this method simply adds an observer to the observers ArrayList
    - Implement the method **removeObserver(…)** : this method simply removes an observer from the observers ArrayList
    - Implement the method **notifyObservers()** : this method iterates the observers ArrayList. For - each **observer** in the **observers** list the method calls the observer's update method and passes the finished pizza as a parameter. {Hint: You can use a regular for loop here}
    - Create a toString() method that returns a string containing the **description** of all the pizzas in the **pizzas** list
    - Create a method **removePizza(…).** This method accepts a **PizzaOrTopping** as a parameter and simply removes that pizza from the pizzas ArrayList.
    - Create a method **addPizza(…).**
        - This method accepts a **PizzaOrTopping** as a parameter and simply adds it to the pizzas ArrayList.
        - This method is also responsible for cooking the pizza. To do this simply insert the

```
Timer pizzaTimer = new Timer();
pizzaTimer.schedule(new TimerTask(){
    public void run(){
        pizza.finish();       // pizza is finished
        finishedPizza = pizza;// save the finished pizza
        removePizza(pizza);   // remove the finished pizza
        notifyObservers();    // notify the observers
    }
},pizza.getCookingTime());
```

        following code into your **addPizza** method.
        {Hint: this creates a thread for each pizza based on the pizzas cooking time therefore each pizza cooks at a different time.}

- Create the **Order** class. The **Order** class should implement the **Observer** interface. The Order class should have three (3) **private** instance variables: **int orderNum**, **boolean collected**, **Subject pizzaOven**.

- Create a constructor that accepts two (2) parameters **int orderNumber** and **Oven pizzaOven**. The constructor should
    - set the **orderNumber**
    - set the **pizzaOven**
    - set **collected** to false.
- Implement the **update(…)** method. The method accepts a **PizzaOrTopping** as a parameter. The method should
    - check if **this.orderNumber** and **pizza.orderNum** are the same. If they are the same then:
        - set the **collected** instance variable to **true** and remove **this** from the pizzaOven's observers list.

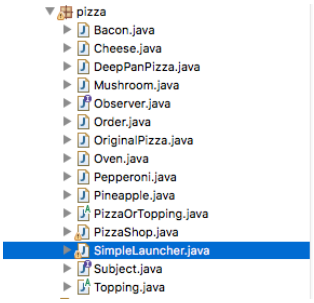
6. You decided to implement the **pizzaShop** class as a **singleton**. The class should have four (4) instance variables **PizzaShop singlePizzaShop**, **int orderCounter, ArrayList<Order> orders**, **Oven pizzaOven**.

- Implement the singleton: {review your notes}.
- The constructor should
    - create **orders**
    - create **pizzaOven**.
    - initialize the **orderCounter** to 1.
- Create a method **void placeOrder(..)** that accepts a **PizzaOrTopping** as a parameter. The method
    - creates a **new Order** object using the **orderCounter** and **pizzaOven** as parameters.
    - increments the **orderCounter**.
    - add the Order to the orders
    - register the order as an observer
    - add the **PizzaOrTopping** parameter to the **pizzaOven**.
- Create the accessor the **getOrderCounter()**.
- Create the accessor the **getPizzaOven()**.
- Create a **toString()** method. The method returns a string containing the description of all pizzas in the **pizzaOven** ArrayList. {Hint: Use a for loop to concatenate the description of the pizzas. getDescription() will help}

Finished…

Having completed your application, you can test it by using a launcher that is given.

Copy the launcher into the same folder as your java files and **run** the launcher.



- SimpleLauncher. {this is text based}

  or

- GUILauncher. {this is graphical}

The launchers have been coded for you and you do not need to alter this code in any way. DO NOT alter the launchers.

Sample output

SimpleLauncher

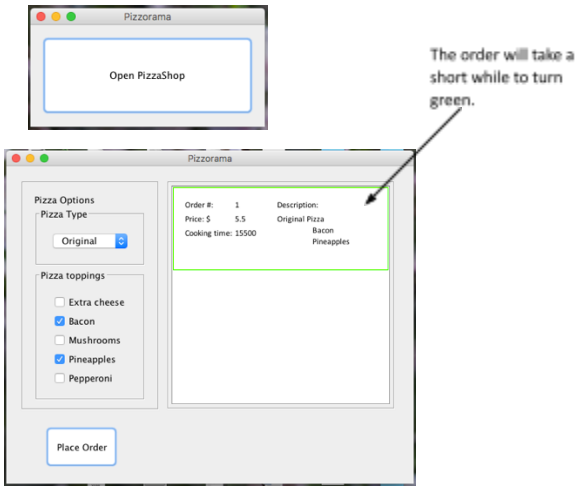
```
*******Pizzashop Management System*******
(1) Order a Pizza:
(2) View Pizzas
(3) Exit the program
*****************************************
1
*******Pizzashop Management System*******
Please choose the type of pizza
(1) Original Pizza:
(2) DeepPan Pizza:
(3) Cancel Order
*****************************************
1
*******Pizzashop Management System*******
Please choose a topping
(1) Pepperoni
(2) Bacon
(3) Pineapples
(4) Mushrooms
(5) Extra Cheese
(6) Confirm Order
(7) Cancel Order
*****************************************
```

```
6
Pizza Ordered: Original Pizza
        Bacon
        Pineapples

*******Pizzashop Management System*******
(1) Order a Pizza:
(2) View Pizzas
(3) Exit the program
*****************************************
```

After running the program the finished pizza will take a short while to appear while to appear.

```
6
Pizza Ordered: Original Pizza
        Bacon
        Pineapples

*******Pizzashop Management System*******
(1) Order a Pizza:
(2) View Pizzas
(3) Exit the program
*****************************************
Pizza is finished Original Pizza
        Bacon
        Pineapples
```

GUILauncher



Submission:

You have to submit **the source code** and **written documentation** via **the LMS**.

The deadline for submission is May 25.

Please compress all your **.java** files and your written documentation into **a single .zip file**.

The zip file must be named strictly following this format:

학번_이름**.zip (e.g., 2026148696_**신우진**.zip)**

Written Documentation:

In the documentation, you should include your program architecture design and a description of implementation methodology as well as an explanation of what you did. **Also, you should include screenshots of your program's output in the document.**

You may include screenshots of the sample inputs as well as different input values that you chose.

Questions & Support:

If you have any questions regarding this assignment, please send an email to shinwoojin@hanyang.ac.kr or visit IT/BT Bld 402-2.

Scoring Criteria:

1. You will get 100 points if the program satisfies all the requested features and runs smoothly.
2. Note: **50 Points for Program code** and **50 Points for the Document**.
3. For missing features, points will be deducted.
4. In case you do not meet the deadline,
    1. 50% of your score will be deducted for a delay within 24 hours
    2. 75% of your score will be deducted for a delay within 48 hours
    3. 0 points will be given for a delay of more than 48 hours.